Final Project: Candy++

Learning Goals

This project will get you more familiar with the following list. Ensure that you are meeting these objectives and have them implemented somewhere in your code.

1. File handling and data loading.
2. Create header and class file implementations.
3. String manipulation.
4. Creative narrative design for candy-based board game.

1 Introduction

In this project, you will design a two-player text-based game involving a candy land environment, racing to cross the finish line first in a terminal board game. The project emphasizes design, creativity, challenging logic, and software engineering. This project has no answer key or set questions. Instead, you'll be given a list of tasks to complete, and it's up to you to fulfill them in the most creative way possible.

We have outlined the critical components of the game below, but there are numerous opportunities for creative expression in how you choose to define the details of your game. There is no CodeRunner that you must match precisely, so take advantage of the flexibility. Whenever we do not explicitly provide details on a given mechanic in the game, please use your own intuition to design an interesting game.

2 Game Play

The following sections provide an overview of how the game is played and what specific components should be considered as the programmer becomes familiar with the architecture code provided.

This is a two-player game that starts with a welcome screen and a printed list of all the characters and their attributes. Each player selects a character from the predefined list of ten characters, as each has their own unique stats (See 2.1). After selecting a character and before the first turn, players should also choose their path type (See 2.2).

The game proceeds with each player alternating in turns. For each turn in the game, the main menu should be displayed (See 2.3). One option is for the user to spin a virtual spinner, allowing for number they landed on to determine how many tiles the player advances on their

turn. Whenever a player decides to move, the board should be displayed with the updated positions.

The board is made up of 52 tiles for each path. The provided board includes two lanes, one for each path (See 2.2). Players can select the same path, and this should be reflected in the board displayed. An example of this is available in Figure 1 and Figure 2. The start of the board is on the far left indicated with a gray starting tile. The end of the board is on the far right indicated with an orange ending tile.

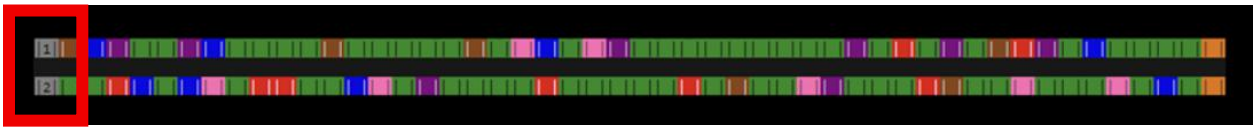

Figure 1. Example of the two-lane board when players choose different path types. Notice that the player numbers are on separate lanes.

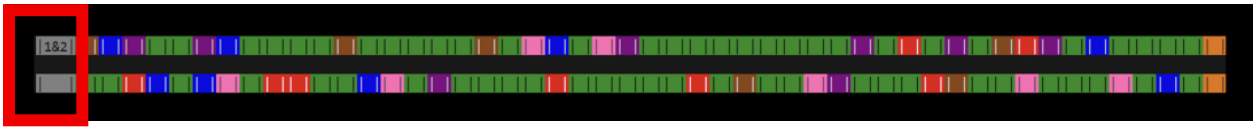

Figure 2. Example of the two-lane board when players choose the same path type. Notice that the player numbers are in the same lane separated using the '&' symbol.

Throughout the board, you'll notice that there are a variety of colored tiles (See 2.4). These colored tiles are randomized using distribution rules, found in gameEngine.cpp. The green tiles are default and have no action, but the other tiles, depending on their color, have a corresponding activity for the player to perform.

The winner is determined once one of the players reaches the end orange tile. The first player to reach the end is not immediately declared the winner though. Whichever player has the larger amount attached to their attribute Focus is declared the Candy++ winner. The output to the terminal should display the winner along with a closing end credits.

2.1 Player Selection Menu

The characters.txt file contains a list of playable characters that players can choose from. Each character includes the following attributes: Focus, Agility, and Luck. Each attribute influences the character's success in the game. All these attributes influence the candy logic functions (See 2.5). Ensure that each character can only be chosen once and update the menu to show only the available characters and their corresponding stats.

2.2 Path Types

After the players select a character, players will also be asked to choose one of two path types. These path types will have an impact on the character's starting stats and the arrangement of tiles they will follow on the game board. Hint: Modify the tile arrangement in board.cpp. The two path options are listed below:

2.2.1 Sugar Rush Path

This path equips the character with +10 agility, +5 luck, and -10 focus. This is perceived as the more aggressive path. While increasing the variables that improve candy logic performance (See 2.5), the focus is decreased, making it potentially more difficult to win the game. Additionally, the board distribution of tiles increases, making it so there are more colored tiles for this path.

2.2.2 Brain Freeze Path

This path equips the character with -10 agility, -5 luck, and +15 focus. This is perceived as the calculated path. While decreasing the variables that support candy logic performance (See 2.5), the focus is increased, making it potentially more likely to win the game. Additionally, the board distribution of tiles decreases, making it so there are fewer colored tiles for this path.

2.3 Main Menu

Provide an interactive menu with at least five options that the players see on their turn. At least two of these options should have secondary layers. The following is an example of what your main menu might look like, but you are encouraged to modify what information you may want to display.

1. Review Player Progress
    a. Print the character's stats
2. Review Character
    a. Print the character's name
3. Review Player's Position
4. Display the Board
5. Move Forward (Use the Virtual Spinner)

2.4 Tile Descriptions

2.4.1 Regular Tiles

Regular tiles are default colored green. For each path, there should be a minimum of twenty regular tiles on the board. This distribution depends on the path (See 2.2).

2.4.2 Special Tiles

These tiles are full of surprises, that take up the remaining of the tiles positioned throughout each lane of the board. These surprises are triggered whenever one of the players lands on the corresponding color. The color and events are described below. Each event has additional details and examples in Section 2.5.

1. Blue Tiles: Candy Jar Sort
2. Pink Tiles: Inventory Scrambler

3. Red Tiles: Combine Candies
4. Brown Tiles: Find Golden Ticket

2.5 Candy Logic Functions

The following four functions are required to trigger whenever a player lands on the corresponding tile color. The exception for this is the luck check. When landing on any of the special tiles, a luck check is performed such as rand() % 100 < luck to determine if the player gets to skip the logic function, or if they must perform it. The other two attributes, agility and focus, should also be integrated into these functions somehow to influence the player's outcome.

The following list is a few items that are up to the programmer to decide how to integrate these functions into the board game:

- How to provide arguments to these functions.
- How/Whether the player will interact with the input and output.
    - Our recommendation: Printing a summary of what the input and output was to the terminal.
- What to do with the output of the functions.

2.5.1 Blue Tiles: Candy Jar Sort

Function	int candyJarSort(Character c, string candies, string targetCandy);
Purpose	Write a function that counts how many times a specific candy (target candy) string appears in the jar (a string of candies separated by commas).
Parameters	Character c, string jar, string targetCandy
Return Value	int – number of target candies found in the jar.

If my candies parameter looked like this: taffy,sour,sweet,chocolate,mike,ike,taffy,taffy

And my targetCandy parameter looked like this: taffy

Then I would expect the following value to be returned: 3

2.5.2: Pink Tiles: Inventory Scrambler

Function	string inventoryScrambler(Character c, string inventory);
Purpose	Write a function that scrambles the parameter inventory by reversing the string and converting every other letter to uppercase.
Parameters	Character c, string inventory
Return Value	string – the scrambled string.

If my inventory parameter looked like this: chocolate

Then I would expect the following value to be returned: eTaLoCoHc

2.5.3: Red Tiles: Combine Candies

Function	string combineCandies(Character c, string candy1, string candy2);
Purpose	Write a function that combines the string parameters to create a new combined string.
Parameters	Character c, string candy1, string candy2
Return Value	string – the combined string created.

If my candy1 parameter looked like this: Sweet

And my candy2 parameter looked like this: Sour

Then I would expect the following value to be returned: SSwoeuert

2.5.4 Brown Tiles: Find Golden Ticket

Function	int findGoldenTicket(Character c, string batch, string ticket);
Purpose	Write a function that focuses on pattern alignment. This function will find the best index that matches the ticket string to the batch string.
Parameters	Character c, string batch, string ticket
Return Value	int – the index of the batch where the highest number of characters matched the ticket.

If my batch parameter looked like this: MatchToTheBestIndex

And my ticket parameter looked like this: best

Then I would expect the following value to be returned: 10

3 Implementation Requirements

This section provides an overview of the files provided to the programmers, programmer task list, the rules regarding group work, the final project timeline, a list of potential extra credit opportunities, and the scoring.

3.1 File Overview

The following table provides a high-level overview of the files that are provided to you. These are the classes that are provided to you to support the architectural design of the final project. However, you are encouraged to add your own classes, additional functions, and change the logic and flow of the program as you see fit.

File Name	Purpose
board.cpp	Source File: Responsible for managing the layout of the tiles and exact positions of each player on the board.
board.h	Header File: Responsible for managing the layout of the tiles and exact positions of each player on the board.
candyLogic.cpp	Source File: Responsible for the focused tasks on string reversal, alternating characters, etc.
candyLogic.h	Header File: Responsible for the focused tasks on string reversal, alternating characters, etc.
character.cpp	Source File: Responsible for initializing the behavior of the character class.
character.h	Header File: Responsible for declaring the behavior of the character class.
characters.txt	CSV File: Ten characters and their attributes.
gameEngine.cpp	Source File: Responsible for managing the interaction between all of the classes. This file contains the main function, recognized as the entry point for the entire game.
loadCharacters.cpp	Source File: Responsible for reading the external data saved in the CSV file using fstream and getline().
loadCharacters.h	Header File: Responsible for reading the external data saved in the CSV file using fstream and getline().
testBoard.cpp	Test File: Contains a board object and functions to test board.cpp.
testCandyLogic.cpp	Test File: Contains a candyLogic object and functions to test candyLogic.cpp.
testLoadCharacters.cpp	Test File: Contains functions to test loadCharacters.cpp.

3.1.1 Compilation Lines

The following is an example of a compile line you can use to test the code provided to you: `g++ -std=c++17 -Wall -Werror -Wpedantic board.cpp testBoard.cpp -o board.exe` (.exe for Window users or .out for Mac users).

3.2 Task List

The following table provides an overview of the tasks expected for programmers to implement.

Task	Pointers
Run the script as is to learn the flow of the architecture provided to you.	See Section 3.1.1.
Debug `board.h`	
Write `character.h`	Refer to `characters.txt`.
Write `character.cpp`	Refer to `character.h`.
Write `loadCharacters.cpp`	Use file input from `characters.txt`. The characters

	should be loaded into the array parameter. Refer to `loadCharacters.h`.
Modify the program to print a welcome screen and display all available characters and their stats.	
Modify the program to allow players to select their character, rather than the default option of random selection.	This random selection is currently implemented in `gameEngine.cpp`.
Modify the program to ensure that each character can only be chosen once. Update the display to show only the available characters and their corresponding stats.	
Modify the program to allow players to select their path type.	It would be useful to display the path type options.
Modify the program to allow players to select different or the same paths. If players select the same path, the board should update.	See Figure 1 and Figure 2.
Modify the program to distribute the colored tiles more often or less frequently depending on the path.	See Section 2.2.
Create a main menu that appears at every turn.	See Section 2.3.
Write `candyLogic.cpp`. In all four candy logic functions, the characters attributes should influence the player's game.	See Section 2.5. Note that this is why `Character c` is passed as a parameter to each function. Refer to `candyLogic.h`.
Write at least one test file.	`testCandyLogic.cpp` and `testLoadCharacters.cpp` are our recommendations. Refer to `testBoard.cpp` for an example.
Modify the program to perform the Luck check, prior to triggering any of the candy logic functions.	See Section 2.5.
Modify the program to ensure that if a player lands on a colored tile, it triggers the corresponding candy logic function.	See Section 2.5.
To conclude the game, the winner should be declared.	We encourage you to be creative in how the winner is displayed.

3.3 Group Work

You are allowed to work with at most one other student who is currently in CSCI 1300 this semester, in any section. We expect that you will both contribute equally to the project. Both group members must submit a zip file for the project. Indicate your partner's name in the comments at the top of each code file. You may also work by yourself if you choose.

If you choose to work in a group, there are additional requirements. If you work in a group, you must also implement a sorting algorithm and apply it to your program, and at least

one other customization (See 3.5). You should not use a library function or any external resources to implement the sorting algorithm.

3.4 Final Project Timeline

- **April 6th**: The final project information is released.
- **Wednesday, April 22 at 11:59 pm**: Submit all files necessary to compile your code as a zip file.
- **Wednesday, April 29**: All interview grading will be completed by this date. You must sign up for an interview grading timeslot between Thursday, April the 23rd and Wednesday the 29th. If you do not sign up or miss your interview, then no points will be awarded for the project.

3.5 Potential Extra Credit Opportunities

Below is a list of some suggestions on how to put your own spin on this project. You are free to make any changes to any of the files to help showcase your creativity. However, note that any changes you make cannot reduce the complexity of the game.

- Game Balancing: Feel free to tweak any numbers in this document to make the game more competitive or interesting.
- Modify the Text File: You can also modify and add to the text file.
- Game Complexity: You can also modify the program in a more creative way to add complexity to the game.
- Change the Theme: The game could take place in an entirely different world or environment.
- Alter the Characters: Instead of candy-based characters, you could have characters from anywhere else with their own attributes.
- Customize the Challenges: Introduce unique challenges in addition to the candy logic.
- Add Unique Interactions: Think about different ways the players can interact with the game, such as special abilities or items.
- Modify the Game Board: Increase the number of tiles or change the tile colors. The players can be symbols instead of numbers.
- Create More Files/Classes/Structs/Functions: You can add more content to represent the complexity you want to implement in your game.
- Creating Test Files: Add additional files specifically to test your code.
- Write to a File: Export information to a file such as the winning character and their ending attribute values.

3.6 Scoring

This final project is worth 10% of your grade. The grading interview is required for a grade.